

אינטגרציית Cisco DNA Center ל-SolarWinds – הסבר

איך `Scripts/DNA.DiscoverNodesAndInterfaces.ps1` מושך את מלאי המכשירים מ-Cisco DNA Center, רושם כל מכשיר כ-Node ב-SolarWinds, ומוסיף לניטור רק את הממשקים שנבחרו.

מסמך מקביל ל-SmartConsole-Integration-Explained.pdf, עבור מסלול ה-DNA.

1. מה האוטומציה הזאת עושה

המטרה היא לקחת את כתובות ה-IP הניהוליות של כל המכשירים הרשומים ב-Cisco DNA Center (שנקרא היום Catalyst Center), לרשום כל אחד מהם כ-Node מנוטר ב-SolarWinds, ואז לגלות את הממשקים של אותו Node ולהוסיף לניטור רק את אלה שעומדים בקריטריון מוגדר.

בניגוד למסלול ה-SmartConsole, שמפוצל לארבעה קבצים, כאן הכול נמצא בסקריפט אחד שרץ מקצה לקצה. הוא עובר שישה שלבים לפי הסדר:

שלב	פונקציה / קטע	תפקיד
1	<code>Get-TokenDNA</code>	מתחבר ל-DNA ב-Basic Auth ומחזיר token לשימוש בכל הקריאות הבאות.
2	<code>Get-DNADevices</code>	מושך את מלאי המכשירים, עמוד אחרי עמוד, ומחזיר את רשימת המכשירים.
3	<code>Connect-Swis + \$components</code>	מתחבר ל-SolarWinds ומתרגם את הכתובות למבנה אחיד עם הגדרות SNMP.
4	<code>Add-Component</code>	יוצר Node חדש ב-SolarWinds ורושם עבורו את ארבעת ה-pollers הסטנדרטיים.
5	<code>Add-DiscoveredInterfaces</code>	מגלה את ממשקי ה-Node, מסנן לפי רשימת היתר, ומוסיף את הנותרים לניטור.
6	<code>Stop-Sessions</code>	סוגר את החיבור ל-SWIS וזורק את ה-token של DNA מהזיכרון.

למה סקריפט אחד, ולא ארבעה כמו במסלול SmartConsole

מסלול ה-SmartConsole מפוצל לפונקציות שנטענות ב-dot-sourcing, עם קובץ CSV אופציונלי בין הגילוי לבין ההוספה — מה שמאפשר שלב סקירה ידני לפני שמשהו נכנס לניטור. הסקריפט הזה בנוי כהרצה אחת רציפה, בלי שלב סקירה: מה שמגיע מ-DNA נכנס לניטור באותה הרצה. זה נוח למשימה מתוזמנת, אבל אומר שטעות בצד ה-DNA מתורגמת מיד ל-Nodes חדשים ולצריכת רישיון.

2. התחברות ל-DNA וקבלת token

הפונקציה `Get-TokenDNA` מבצעת `POST` אל `/dna/system/api/v1/auth/token` עם אימות Basic: שם המשתמש והסיסמה משורשרים בפורמט `user:pass`, מקודדים ב-Base64 ונשלחים בכותרת `Authorization`.

```
$pair = "${user}:${pass}"
$encode = [System.Convert]::ToBase64String([text.encoding]::ASCII.GetBytes($pair))
$header = @{
    "Content-Type" = "application/json"
    "Authorization" = "Basic $encode "
}
$response = Invoke-WebRequest -Uri $uri -Headers $header -Method Post -SkipCertificateCheck
return $response.Content | ConvertFrom-Json
```

הקריאה עטופה ב- `try/catch`, ומיד אחריה נבדק שה-token שחזר אינו ריק. שתי הבדיקות נחוצות בנפרד: שרת שמחזיר 200 עם גוף לא צפוי לא יזרוק שגיאה, ובלי הבדיקה המפורשת הסקריפט היה ממשיך עם token ריק ונכשל רק בשלב הבא, עם הודעה מטעה.

```
try {
    $tokenObj = Get-TokenDNA -uri $dnaUrlToken -user $dnaCredentials.UserName ...
    $tokenString = $tokenObj.Token

    if (-not $tokenString) {
        Write-Error "DNA authentication did not return a token. Check URL/credentials."
        Stop-Sessions
        exit
    }
} catch {
    Write-Error "Failed to authenticate with DNA: $($_.Exception.Message)"
    Stop-Sessions
    exit
}
```

3. משיכת מלאי המכשירים

דורשות — ומושכת את רשימת המכשירים. `Get-DNADevices` שולחת את ה-token בכותרת `x-auth-token` — הכותרת שכל קריאות ה-API של DNA

ה-API מחזיר עמוד אחד בכל קריאה, ולא את כל המלאי. לכן הפונקציה רצה בלולאה עם `offset` ו-`limit` וממשיכה כל עוד העמוד שחזר מלא. כשעמוד חוזר קצר מגודל העמוד — זהו העמוד האחרון.

```
$header = @{
    "Content-Type" = "application/json"
    "x-auth-token" = $token
}

$devices = @()
$pageSize = 500
$offset = 1 # the DNA device inventory is 1-based

do {
    $pageUri = "($url)?offset=$((offset))&limit=$((pageSize))"
    $response = Invoke-WebRequest -Uri $pageUri -Headers $header -Method Get -SkipCertificateCheck
    $page = @(($response.Content | ConvertFrom-Json).response)

    $devices += $page
    $offset += $pageSize
} while ($page.Count -eq $pageSize)

return $devices
```

למה זה קריטי דווקא כאן

קריאה בודדת בלי pagination לא מחזירה שגיאה — היא פשוט מחזירה את העמוד הראשון בלבד. בסביבה עם יותר מכשירים מגודל העמוד, חלק מהמכשירים לא היו נרשמים לניטור ושום הודעה לא הייתה מופיעה. זו הסיבה שהלולאה נבנתה לפי אורך העמוד ולא לפי שדה `total`.

4. התחברות ל-SWIS ובניית רשימת הרכיבים

אחרי שהמלאי בידיים, הסקריפט מוציא ממנו את הכתובות הניהוליות ומוודא שיש מה לעבד לפני שהוא בכלל נוגע ב-SolarWinds:

```
$addresses = @($results.managementIpAddress)

if (-not $addresses) {
    Write-Warning "DNA returned no devices - nothing to process."
    Stop-Sessions
    exit
}
```

רק אז נפתח החיבור ל-SWIS, ולאחריו כל כתובת מתורגמת ל-hashtable אחיד שמחזיק, לצד הכתובת, את הגדרות ה-SNMP המשותפות שהגיעו מבלוק ה-`param`:

```
$components = $addresses |
    Where-Object { $_ } |
    ForEach-Object { $_.Split(',') } |
    ForEach-Object { $_.Trim() } |
    Where-Object { $_ } |
    ForEach-Object {
        @{
            IPAddress    = $_
            EngineID     = $EngineID
            SNMPVersion  = $SNMPVersion
            DNS           = ""
            SysName      = ""
            Community    = $SNMPCommunity
        }
    }
}
```

הפיצול ב-`Split(',')` נשאר מהמסלול הקודם: הוא מאפשר גם ערך יחיד וגם מחרוזת עם כמה כתובות מופרדות בפסיק, כך שאותו בלוק היה עובד גם על רשימה שהוזנה ידנית.

5. הוספת Node ורישום pollers

יוצרת את ה-Node בשתי קריאות SWIS. הראשונה יוצרת את האובייקט ומחזירה URI בלבד; כדי לקבל את ה-`NodeID` — שנדרש לרישום ה-pollers — צריך לשלוח את האובייקט בחזרה בקריאה שנייה.

```

$newNodeUri = New-SwisObject -SwisConnection $swis -EntityType "Orion.Nodes" -Properties
$newNodeProps
$nodeProps = Get-SwisObject -SwisConnection $swis -Uri $newNodeUri

# Register the standard set of pollers for the node
$poller = @{
    NetObject      = "N:" + $nodeProps["NodeID"]
    NetObjectType   = "N"
    NetObjectID     = $nodeProps["NodeID"]
}

foreach ($pollerType in @(
    "N.Status.ICMP.Native",
    "N.ResponseTime.ICMP.Native",
    "N.Details.SNMP.Generic",
    "N.Uptime.SNMP.Generic"
)) {
    $poller["PollerType"] = $pollerType
    New-SwisObject -SwisConnection $swis -EntityType "Orion.Pollers" -Properties $poller | Out-Null
}

return $nodeProps["NodeID"]

```

ארבעת ה-pollers הם המינימום שהופך Node חדש למנוטר בפועל: זמינות והיענות ב-ICMP, ופרטי מכשיר וזמן פעילות ב-SNMP. בלי רישום מפורש שלהם ה-Node ייווצר, אבל יישאר ריק בממשק.

6. גילוי הממשקים והסינון

קוראת ל-verb של NPM שמבצע גילוי ממשקים על ה-Node, ורק אם הגילוי הצליח ממשיכה לסינון. `Add-DiscoveredInterfaces`

הסינון הוא **רשימת היתר**: נשמרים רק ממשקי TenGigabit ו-Port-channel שנמצאים במצב תפעולי תקין (`ifOperStatus` שווה 1). כל השאר מוסר לפני ההוספה.

```

@($discovered.DiscoveredInterfaces.DiscoveredLiteInterface) | Where-Object {
    $_.Caption.InnerText -notmatch '^(TenGigabitEthernet|TenGigE|Te\d|Port-channel|Po\d)' -or
    $_.ifOperStatus -ne '1'
} | ForEach-Object { $discovered.DiscoveredInterfaces.RemoveChild($_) | Out-Null }

```

שלוש נקודות שקל לטעות בהן בבולוק הזה

- **התנאי הפוך בכוונה.** הבולוק מסיר את מה ש-`Where-Object` מחזיר, ולכן "השאר רק A ו-B שהם תקינים" נכתב כ-`-notmatch` על השם או `-ne '1'` על הסטטוס. שימוש ב-`-and` במקום `-or` היה משאיר בטעות כל ממשק תקין שאינו ברשימת ההיתר.
- **השוואה מדויקת ולא regex.** `-ne '1'` ולא `-match '1'`, כי `-match` על מספר מחפש תת-מחרוזת ולכן היה מתאים גם ל-12 ול-21.
- **מימוש הרשימה ב-`@()` לפני הלולאה.** `RemoveChild` מקצר את ה-`XmlNodeList` החי בזמן שה-pipeline עדיין עובר עליו, וללא המימוש המוקדם חלק מהממשקים היו מדולגים.

לאחר הסינון נספרים הממשקים שנשארו. אם לא נשאר אף אחד, הפונקציה יוצאת בלי לקרוא ל-API; אחרת ההוספה עצמה עטופה ב-`try/catch`, והודעת ההצלחה מודפסת רק כשהקריאה באמת הצליחה:

```

$interfaceCount = @($discovered.DiscoveredInterfaces.DiscoveredLiteInterface).Count

if ($interfaceCount -eq 0) {
    Write-Host " No interfaces left to add for node $($nodeId) after filtering." -ForegroundColor
DarkBlue
    return
}

try {
    Invoke-SwisVerb $swis Orion.NPM.Interfaces AddInterfacesOnNode `
        @($nodeId, $discovered.DiscoveredInterfaces, "AddDefaultPollers") | Out-Null
    Write-Host " Added $interfaceCount interface[s] for node $($nodeId)." -ForegroundColor Green
} catch {
    Write-Host " Failed to add interfaces for node $($nodeId): $($_.Exception.Message)" -
ForegroundColor Red
}

```

7. הלולאה הראשית

הלולאה עוברת על הרכיבים, בודקת מול SWIS אם ה-Node כבר קיים, ומוסיפה רק אם לא. בשני המקרים — חדש או קיים — היא ממשיכה לגילוי הממשקים, כך שגם Node ותיק מקבל את הממשקים שהתווספו אליו מאז.

```

foreach ($component in $components) {
    Write-Host "Processing component $($component.IPAddress)..."

    try {
        $existing = Get-SwisData -SwisConnection $swis `
            -Query "SELECT NodeID FROM Orion.Nodes WHERE IPAddress = '$($component.IPAddress)'"

        if ($existing) {
            $nodeId = $existing
            Write-Host " Node already exists {NodeID $nodeId}, skipping add." -ForegroundColor
DarkBlue
        }
        else {
            $nodeId = Add-Component $component
            Write-Host " Added node [NodeID $nodeId]."

            Add-DiscoveredInterfaces $nodeId
        }
    } catch {
        Write-Host " Failed to process $($component.IPAddress): $($_.Exception.Message)"
    }
}

```

8. סגירת הסשנים

נקראת גם בסיום התקין וגם לפני כל אחת מנקודות היציאה המוקדמות, כדי ששגיאה בשלב ה-DNA `Stop-Sessions` לא תדלג על הניקוי.

```

if ($script:tokenString) {
    $script:tokenObj = $null
    $script:tokenString = $null
    Write-Host "Discarded the DNA API token." -ForegroundColor Green
}

if ($script:swis) {
    try {
        $script:swis.Close()
        Write-Host "Closed the SWIS connection." -ForegroundColor Green
    } catch {
        Write-Warning "Failed to close the SWIS connection: $($_.Exception.Message)"
    }
    $script:swis = $null
}

```

שני הצדדים אינם סימטריים

DNA: ל-Catalyst Center אין endpoint לביטול token. הטוקן תקף שעה ופג מעצמו, ולכן בניגוד ל-logout האמיתי שמבצע סקריפט ה-SmartConsole, כאן אפשר רק לזרוק אותו מהזיכרון.

SolarWinds: אין cmdlet בשם `Disconnect-Swis`, אבל `Connect-Swis` מחזיר אובייקט `InfoServiceProxy` שמממש `IDisposable` – ולכן הסגירה בצד הזה אמיתית.

בהרצה חד-פעמית של קובץ `.ps1` המשתנים מתים ממילא בסיום והתהליך סוגר את החיבור. הערך המעשי של הבלוק הוא בהרצה עם dot-sourcing מתוך runner ארוך-חיים.

9. זרימה מקצה לקצה

שלב	מה קורה	אם זה נכשל
1	קבלת token מ-DNA	שגיאה, <code>Stop-Sessions</code> , יציאה
2	משיכת המלאי, עמוד אחר עמוד	שגיאה, <code>Stop-Sessions</code> , יציאה
3	חילוץ הכתובות הניהוליות	אזהרה אם אין מכשירים, ויציאה
4	התחברות ל-SWIS	שגיאה, <code>Stop-Sessions</code> , יציאה
5	לכל כתובת: הוספת Node אם אינו קיים	נרשם ללוג, ממשיך לכתובת הבאה
6	גילוי, סינון והוספת ממשקים	נרשם ללוג, ממשיך לכתובת הבאה
7	סגירת הסשנים	אזהרה בלבד

ההבדל בין שלבים 1-4 לשלבים 5-6 מכון: כשל בהתחברות עוצר הכול, כי בלעדיו אין מה לעשות; כשל על מכשיר בודד רק נרשם ללוג, כדי שמכשיר אחד בעייתי לא יעצור את כל ההרצה.

10. מה לשנות לפני הרצה אמיתית

- למלא את פרטי החיבור. `$hostname`, `$username`, `$dnaServer` ונתיב ה-clixml של אישורי ה-DNA (`$dnaCredentials`) מגיעים ריקים ומחכים לעריכה. הנתיב הריק של אישורי ה-DNA יזרוק שגיאה מיד בהרצה.

- **להגדיר community אמיתי.** `$SNMPCommunity` ריק כברירת מחדל; Node שייוצר עם community ריק לא יצליח לבצע polling ב-SNMP.
- **לאמת את ה-EngineID.** ברירת המחדל היא 2; בסביבה עם polling engine יחיד הערך הנכון הוא בדרך כלל 1.
- **לאמת את ה-endpoint ואת ה-pagination.** הסקריפט פונה ל- `network-devices`, בעוד שה-endpoint המתועד ברוב הגרסאות הוא `network-device`. כדאי לוודא מול הגרסה שרצה אצלכם שגם השם וגם הפרמטרים `limit / offset` נתמכים.
- **PowerShell 7 ומעלה.** `-SkipCertificateCheck` אינו קיים ב-PowerShell 5.1, ושם הסקריפט ייפול.
- **להריץ קודם על קבוצה קטנה.** אין שלב סקירה ואין הרצה יבשה — כל מכשיר שמגיע מ-DNA נרשם לניטור וצורך רישיון.

11. מגבלות ידועות

דברים שעדיין פתוחים בקוד, ונרשמים כאן כדי שלא יתגלו בהפתעה:

- **אין סינון לפי זמינות.** כל מכשיר שמופיע ב-DNA נרשם, גם אם ה- `reachabilityStatus` שלו אינו תקין — מה שעלול לייצר גל התראות ביום הראשון.
- **המטא-דאטה של DNA לא מנוצלת.** מתוך כל מה ש-DNA מחזיר נלקחת רק `managementIpAddress`, והשדות `DNS` ו- `SysName` נשארים ריקים, למרות ש-DNA מספק גם `hostname`, `platformId`, `family` ו- `serialNumber`.
- **אין סינון כפילויות.** אם DNA מחזיר את אותה כתובת פעמיים, היא תעובד פעמיים.
- **שאלת ה-SWQL בנויה בשרשור מחרוזות ולא כשאלתה פרמטרית.**
- **תת-ממשקים נכללים.** תבנית הסינון בודקת את תחילת השם בלבד, ולכן `TenGigabitEthernet1/0/1.100` ייכלל גם הוא.
- **כפילות מול סקריפט ה-SmartConsole.** `Add-Component`, הלולאה הראשית ובניית רשימת הרכיבים כמעט זהים בשני הקבצים, ותיקון באחד אינו מגיע לשני.

מסמך זה מתאר את `Scripts/DNA.DiscoverNodesAndInterfaces.ps1` שבמאגר. קטעי הקוד מצוטטים מהסקריפט כפי שהוא, ומקוצרים במקומות שבהם הושמטו הערות או שורות שאינן נחוצות להסבר. המסמך לא נבדק מול הרצה חיה של הסקריפט.